

Table of Contents

Chapter 6: Data and Analysis	6
Complete Study Guide — Grade 12 Computer Science	6
Written in the style of Prof. David J. Malan (Harvard CS50)	6
Student Learning Outcomes	6
Introduction	7
The Hook: A Map That Saved Thousands of Lives	7
6.1 Introduction to Data Science	7
What is Data Science? The Data-to-Insight Pipeline	7
The Hook	7
The Explanation	7
The Data Science Workflow	8
Pause & Think — 6.1	8
Quick Recap	9
6.2 Understanding Data	9
Types of Data: Structured vs. Unstructured	9
The Hook	9
The Explanation	9
Structured Data	9
Unstructured Data	10
Key Comparison	10
Quick Recap	11
Data Sources and Data Collection Methods	11
The Explanation	11
Common Data Sources	11
Manual vs. Automatic Collection	11
Real-Time vs. Stored Data	11
Pause & Think — 6.2a	12
Quick Recap	12
Data Storage and Management Concepts	12
The Explanation	12
Data Storage Options	12
Key Data Management Concepts	12
Quick Recap	13
6.3 Overview of Machine Learning	13
Learning from Data: Moving from Rules to Patterns	13

The Hook: Arthur Samuel and the Game of Checkers (1959).....	13
The Explanation.....	14
Why Does This Matter?	14
The Practical Walkthrough: Traditional Programming vs. Machine Learning	14
Traditional Programming	14
Machine Learning	15
The Chef Analogy	15
Pause & Think — 6.3.....	15
Quick Recap	15
6.4 Machine Learning Mechanisms.....	16
Three Ways a Machine Can Learn	16
The Hook: Three Ways Students Learn	16
Supervised Machine Learning	16
The Explanation.....	16
What is Labelled Data?	17
How Supervised Learning Works (Step by Step)	17
Two Main Types of Supervised Learning Tasks.....	17
Real-World Applications of Supervised Learning.....	17
Pause & Think — 6.4a.....	18
Quick Recap	18
Unsupervised Machine Learning	18
The Explanation.....	18
What is Unlabelled Data?	18
The Most Common Unsupervised Technique: Clustering.....	18
Real-World Applications of Unsupervised Learning.....	19
Pause & Think — 6.4b.....	19
Quick Recap	19
Reinforcement Machine Learning.....	19
The Explanation.....	19
The Dog Training Analogy	20
The Reinforcement Learning Cycle	20
Real-World Applications of Reinforcement Learning	20
Comparison of All Three Mechanisms	21
Pause & Think — 6.4c	21
Quick Recap	21
6.5 Applications of Machine Learning.....	21
The Hook: The \$1 Million Netflix Prize (2006–2009)	21
Use of Machine Learning in Healthcare	22
The Explanation.....	22

Use of Machine Learning in Business	23
The Explanation.....	23
Use of Machine Learning in Education	25
The Explanation.....	25
Everyday Applications of Machine Learning	26
The Explanation.....	26
Grab a Partner — 6.5	26
Quick Recap	27
6.6 Building a Machine Learning Model	27
The Practical Process: From Raw Data to Predictions.....	27
The Hook	27
Selecting Features and Target Values	27
The Explanation.....	27
Example: Student Performance Prediction.....	28
Why Feature Selection Matters	28
Quick Recap	29
Feature Engineering Basics	29
The Explanation.....	29
Step 1: Handling Missing Values (Null/Empty Cells)	29
Step 2: Encoding Categorical Data (One-Hot Encoding)	29
Step 3: Feature Scaling	30
Step 4: Creating New Features	30
Pause & Think — 6.6a.....	31
Quick Recap	31
Train-Test Split of Data	31
The Explanation.....	31
The Exam Paper Analogy	32
How to Split.....	32
Why 80/20?.....	32
Overfitting vs. Underfitting	33
The Python Code: Train-Test Split.....	34
Quick Recap	35
Building a Simple Predictive Model	35
The Complete Python Walkthrough.....	35
Pause & Think — 6.6b.....	37
Quick Recap	38
6.7 Model Evaluation and Performance Metrics.....	38
Why Accuracy Alone Can Be Dangerous	38

The Hook: The Spam Wars (1990s)	38
The Confusion Matrix: Your Evaluation Foundation	38
Accuracy and Error Rate	39
The Explanation.....	39
Why Accuracy Can Be Misleading: The Imbalanced Data Problem	40
Quick Recap	40
Precision and Recall.....	40
The Explanation.....	40
Precision.....	40
Recall.....	41
The Precision-Recall Trade-Off	42
F1-Score	42
The Explanation.....	42
When to Use Which Metric	44
Grab a Partner — 6.7	44
Quick Recap	44
6.8 Validation and Model Improvement	44
The Hook: Getting the Formula Right.....	44
Validation Dataset Preparation	45
The Explanation.....	45
Practical Tip: K-Fold Cross-Validation	46
Quick Recap	46
Model Robustness and Reliability	46
The Explanation.....	46
Quick Recap	47
Improving Models Using Hyperparameter Tuning	47
The Explanation.....	47
The Speaker Volume Analogy	48
Practical Example: Decision Tree Depth	49
Pause & Think — 6.8.....	50
Quick Recap	50
6.9 Predictive Modeling and Causality	50
The Hook: Ice Cream and Drowning	50
Predictive Outcomes in Machine Learning	50
The Explanation.....	50
Understanding Causality	51
The Explanation.....	51
How to Establish Causation.....	52

Difference Between Prediction and Causation	52
The Practical Walkthrough.....	52
Real-World Importance.....	53
Pause & Think — 6.9.....	53
Quick Recap	53
6.10 Machine Learning Tools and Platforms	54
Choosing Your Toolkit	54
The Hook: The Right Tool for the Right Job.....	54
Using Excel for Data Analysis.....	54
The Explanation.....	54
What Excel Does Well	54
Practical Example: Analysing Student Data in Excel.....	55
Excel Limitations.....	55
Quick Recap	55
Introduction to R for Data Science.....	55
The Explanation.....	55
What R Does Well	56
A Simple R Example	56
Quick Recap	56
Python and Jupyter Notebooks for Machine Learning.....	57
The Explanation.....	57
Python's Key Libraries for Data Science	57
What is Jupyter Notebook?	57
A Complete Python Data Science Workflow.....	59
Python vs. R vs. Excel — When to Use What	61
Pause & Think — 6.10.....	61
Quick Recap	62
Chapter Summary	62
Key Vocabulary Glossary	65
Exercises	68
Multiple Choice Questions.....	68
Short Answer Questions	69
Long Questions	70
Quick Review Cards	75

Chapter 6: Data and Analysis

Complete Study Guide — Grade 12 Computer Science

Written in the style of Prof. David J. Malan (Harvard CS50)

A Note to You Before We Begin

You are about to learn one of the most powerful skills of the 21st century: turning raw numbers and messy data into real knowledge. Every recommendation YouTube makes. Every spam email your inbox blocks. Every time a doctor uses a scan to detect cancer early — that is data science and machine learning at work.

You do not need to be a maths genius. You do not need to memorise every formula. You need curiosity, clear thinking, and the willingness to experiment. That is exactly what this chapter builds.

Let's begin.

Student Learning Outcomes

By the end of this chapter, you will be able to:

1. Explain what data science is and why it matters in the modern world.
 2. Identify and distinguish between structured and unstructured data.
 3. Describe the difference between traditional programming and machine learning.
 4. Explain three types of machine learning: supervised, unsupervised, and reinforcement.
 5. List real-world applications of machine learning in healthcare, business, education, and daily life.
 6. Walk through the steps of building a simple machine learning model.
 7. Calculate and interpret accuracy, precision, recall, and F1-score.
 8. Explain what validation, overfitting, and hyperparameter tuning mean.
 9. Distinguish between prediction and causation using real examples.
 10. Identify and compare tools used in data science: Excel, R, and Python/Jupyter.
-

Introduction

The Hook: A Map That Saved Thousands of Lives

It is 1854. London. Hundreds of people are dying from cholera — a deadly disease — in the Soho neighbourhood. No one knows why. Doctors believe the disease spreads through bad air.

Then a physician named **John Snow** does something radical for his time: he collects data.

He marks every single cholera death on a paper map of the streets. He notes the location of every public water pump. When he steps back and looks at his map, a pattern jumps out immediately. Nearly every death clusters tightly around **one pump** on Broad Street.

He removes the handle from that pump. The deaths stop almost immediately.

John Snow did not have a computer. He did not have Python or Excel. But he used **data collection**, **pattern recognition**, and **evidence-based reasoning** to solve a crisis that cost thousands of lives.

That is data science. And that story is from 1854.

Today, the same process — collect data, find patterns, make decisions — runs inside your phone, your hospital, your bank, and your favourite streaming app. This chapter teaches you how it works.

6.1 Introduction to Data Science

What is Data Science? The Data-to-Insight Pipeline

The Hook

Imagine you are the manager of a school. You have attendance records, quiz scores, assignment grades, and survey feedback from 500 students over three years. You have a lot of data. But data alone tells you nothing.

Data science is the process of turning that raw pile of numbers and text into **useful knowledge** — patterns, predictions, and decisions that help you run your school better.

The Explanation

Data science means studying data to find patterns and useful information. It is not just one skill. It combines:

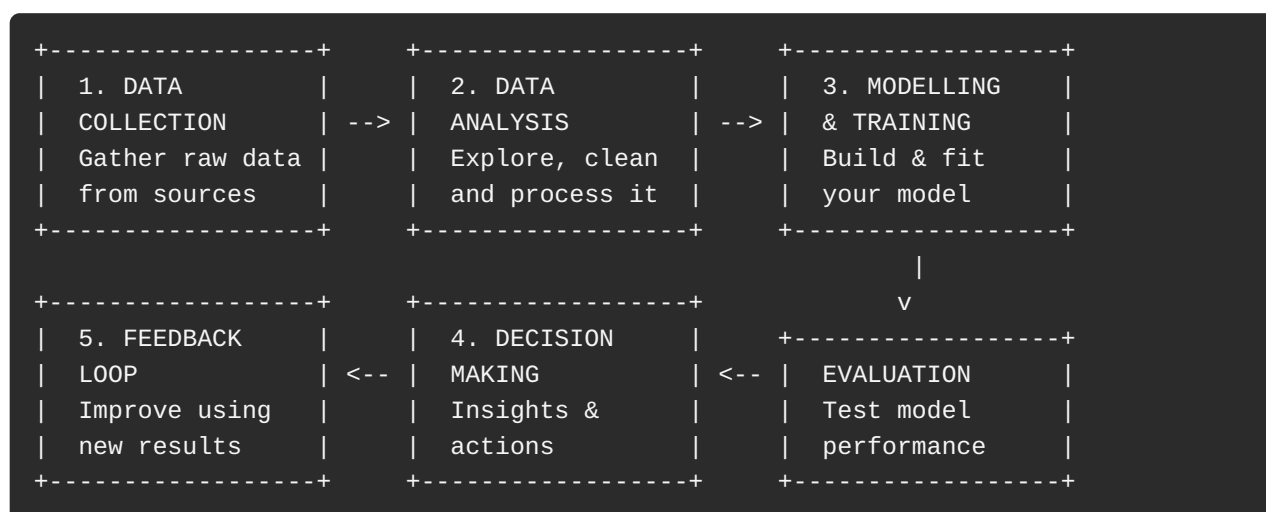
- **Mathematics** — working with numbers and calculations
- **Statistics** — summarising and comparing data

- **Computer Science** — writing code and using tools
- **Domain Knowledge** — understanding the subject you are studying (medicine, business, education, etc.)
- **Communication** — explaining your findings clearly to others

Because it combines so many fields, data science is called **interdisciplinary** (inter = between, disciplinary = fields of study).

The Data Science Workflow

Every data science project follows a similar pipeline. Think of it like an assembly line:



Step 1 — Data Collection: You gather raw data from sources (surveys, sensors, databases, websites).

Step 2 — Data Analysis (Exploring & Cleaning): You look at the data, fix errors, handle missing values, and understand what you have.

Step 3 — Modelling & Training: You build a machine learning model and teach it using your cleaned data.

Step 4 — Decision Making: You use the model's output (insights, predictions) to take actions.

Step 5 — Feedback Loop: You check if your decisions worked. You use new results to improve the model. The cycle repeats.

TIDBIT: Many organisations use data science to support decision-making. It helps them understand trends and hidden patterns that would be invisible without analysis.

Pause & Think — 6.1

Think of a school canteen that sells food to students. List **three types of data** the canteen manager could collect. Then describe **one useful insight** that data could reveal. Share your answer with a partner.

(Example answer: Number of meals sold each day → insight: which days have the most customers, so the manager can order the right amount of ingredients.)

Quick Recap

Data science is the process of collecting, cleaning, analysing, and interpreting data to find patterns and support decision-making. It is interdisciplinary — it needs maths, coding, statistics, and real-world knowledge working together.

6.2 Understanding Data

Types of Data: Structured vs. Unstructured

The Hook

Imagine two students submit their homework. Student A submits a neat table with columns: Name, Age, Score, Grade. Every row is perfectly organised. Student B submits a voice note recording their answer, three photos of handwritten notes, and a long paragraph of text with no structure at all.

Both students gave you "data." But the first is easy to search and analyse. The second requires special effort to process.

This is exactly the difference between **structured** and **unstructured** data.

The Explanation

Data is a collection of raw facts and values. It can be numbers, text, images, audio, or video. Data is produced every day through millions of human and machine activities.

Structured Data

Structured data is organised in a clear, fixed format — rows and columns, like a spreadsheet or a database table.

```
+-----+-----+-----+-----+-----+
| ID    | Name   | Age | Score | Grade |
+-----+-----+-----+-----+-----+
| 001   | Aisha  | 17  | 85    | A     |
| 002   | Bilal  | 18  | 72    | B     |
| 003   | Fatima | 17  | 91    | A+    |
| 004   | Hamza  | 19  | 63    | C     |
+-----+-----+-----+-----+-----+
```

Each row is one record. Each column is one attribute. Every value fits neatly into its cell. This kind of data is easy to search, filter, sort, and feed into machine learning models.

Examples of structured data:

- Student mark sheets
- Bank transaction records
- Hospital patient databases
- Product inventory spreadsheets

Unstructured Data

Unstructured data does not follow a fixed format. It comes in free-form shapes that do not fit neatly into rows and columns.

UNSTRUCTURED DATA EXAMPLES:

+-----+-----+	
Email text	Images / Photos
"Hi, I wanted to ask	[Scan of patient X-ray]
about my account..."	[Instagram photo]
+-----+-----+	
Audio recordings	Video files
[Customer phone call]	[CCTV footage]
[Podcast episode]	[YouTube video]
+-----+-----+	

Unstructured data is **much harder to analyse** directly. You cannot put a photograph into a spreadsheet cell and run a formula. Special tools — like image recognition, speech-to-text, and natural language processing — are needed to process it.

Did You Know? More than **80% of the world's data is unstructured**. This includes social media posts, emails, videos, voice messages, and documents. Processing unstructured data is one of the biggest challenges — and biggest opportunities — in modern data science.

Key Comparison

+-----+-----+-----+		
Feature	Structured Data	Unstructured Data
+-----+-----+-----+		
Format	Fixed (rows & columns)	Free-form (any shape)
Examples	Spreadsheets, databases	Images, audio, video, text
Easy to search?	Yes	Not directly
Easy to analyse?	Yes	Needs special tools
Storage	Relational databases	Data lakes, file systems
+-----+-----+-----+		

Quick Recap

Structured data fits into neat tables and is easy to analyse. Unstructured data — like photos, audio, and text — does not follow a fixed format and needs special tools to process. Both types are important in modern data science.

Data Sources and Data Collection Methods

The Explanation

Before you can analyse anything, you need to **collect** data. Data comes from many sources and can be collected in many ways.

Common Data Sources

Source	Example
Surveys & Questionnaires	Student satisfaction form
Interviews	Recorded conversations with customers
Online Forms	Registration or feedback forms
Sensors & IoT Devices	Temperature sensors, smart meters
Social Media Platforms	Tweets, Facebook posts, Instagram likes
Government Databases	Census data, public health records
Transaction Systems	Bank records, e-commerce purchase logs
Web Scraping	Automated collection from websites

Manual vs. Automatic Collection

Manual data collection uses human effort — someone fills in a form, counts stock items, or writes down observations. It is slow and can have human errors.

Automatic data collection uses machines and software — a sensor records temperature every minute, a website logs every click. It is fast and can capture massive amounts of data.

Real-Time vs. Stored Data

- **Real-time collection:** Data is captured and processed immediately (e.g., a live GPS tracker, live stock prices).
- **Batch collection:** Data is saved over time and analysed later (e.g., monthly sales reports, end-of-term exam results).

Key Point: The quality of your analysis depends directly on the quality of your data collection. Garbage in = Garbage out. Poor data leads to incorrect conclusions, no matter how sophisticated your model is.

Pause & Think — 6.2a

You want to study whether students who sleep more than 8 hours per night perform better in exams.

List: (a) two data sources you would use, (b) one manual and one automatic collection method, and (c) one potential problem with your data collection.

Quick Recap

Data can be collected from surveys, sensors, online platforms, and databases. Collection can be manual or automatic, real-time or batched. Good data collection is the foundation of any reliable analysis.

Data Storage and Management Concepts

The Explanation

Once data is collected, it needs to be **stored safely** and **managed effectively** so it can be retrieved and used later.

Data Storage Options

Storage Type	Description & Example
Files	Simple text files, CSV files on a hard drive
Relational Databases	Organised tables linked together (e.g., MySQL)
Cloud Storage	Online storage (e.g., Google Drive, AWS S3)
Data Warehouses	Large-scale systems for historical analysis
Data Lakes	Store raw data in any format until needed

Key Data Management Concepts

Data Organisation: Keeping data in logical, searchable structures (folders, tables, named columns).

Data Security: Protecting data from unauthorised access using passwords, encryption, and access controls.

Data Backup: Creating copies of data so it is not lost if a system fails. The "3-2-1 rule" says: keep 3 copies, on 2 different media, with 1 copy offsite.

Data Integrity: Ensuring data stays accurate and consistent over time. No accidental changes, no corruption.

Data Cleaning (a critical step): Real-world data is almost never perfect. Before analysis, you must handle:

COMMON DATA QUALITY PROBLEMS:

Problem	Example
Missing values (nulls)	A student's attendance column is blank
Duplicate records	Same student recorded twice in the table
Incorrect data types	Age stored as "seventeen" not 17
Outliers	A score of 150 out of 100 (impossible)
Inconsistent formatting	Date as "01/07/2025" vs "July 1, 2025"

HOW TO HANDLE MISSING VALUES:

- Option 1: Delete the row with missing data (if very few rows are affected)
- Option 2: Fill in the missing value with the column average (mean imputation)
- Option 3: Fill with the most common value (mode imputation)
- Option 4: Use a model to predict what the missing value should be

Did You Know? Data scientists spend approximately **60–80% of their total project time** just cleaning and preparing data. Getting clean data is not glamorous, but it is the most important step.

Quick Recap

Data must be stored in organised systems (files, databases, cloud), protected with security measures, backed up regularly, and cleaned before analysis. Handling missing, duplicate, or incorrect data is a critical step before any model can be built.

6.3 Overview of Machine Learning

Learning from Data: Moving from Rules to Patterns

The Hook: Arthur Samuel and the Game of Checkers (1959)

In 1959, an IBM engineer named **Arthur Samuel** had a curious idea. What if instead of writing a program that knew all the rules of checkers, he wrote a program that could **learn** to play checkers by playing thousands of games?

He did exactly that. He wrote a program that played checkers against itself, over and over. Every time it lost, it updated its strategy. Every time it won, it reinforced what worked.

The result? The program eventually got good enough to **beat Arthur Samuel himself**.

He called what the program was doing "**Machine Learning**" — and that name stuck. Forever.

That was 1959. Today, the same concept powers self-driving cars, medical diagnosis systems, and the algorithm that decides what you see next on YouTube.

The Explanation

Machine learning is a subfield of data science that enables computer systems to learn from data and improve their performance **without being explicitly programmed with rules for every situation**.

Instead of a programmer writing every rule ("IF email contains FREE MONEY, THEN mark as spam"), the system **studies thousands of examples** and discovers the rules itself.

Why Does This Matter?

Some problems are too complex for humans to write every rule manually. Consider:

- **Face recognition:** How do you write a rule to recognise your friend's face? There are too many variations in lighting, angle, and expression.
- **Medical diagnosis:** How do you write a rule to detect cancer in a scan? Doctors learn by studying thousands of real cases — and so do machine learning models.
- **Language translation:** How do you write rules for translating every sentence in every human language? You cannot. But a model trained on millions of translated sentences can learn to do it.

The Practical Walkthrough: Traditional Programming vs. Machine Learning

Traditional Programming

TRADITIONAL PROGRAMMING:

```
Fixed Rules (written by programmer) ---> |         |
                                           |  Program  | ---> Output
Input Data -----> |         |
                                           +-----+
```

Example: Sorting Student Grades

```
Rules: IF score >= 90, THEN grade = "A"
       IF score >= 80, THEN grade = "B"
       IF score >= 70, THEN grade = "C"
```

Input: Score = 85

Output: Grade = "B"

The programmer writes every rule. The program follows them exactly.
What happens when the rules change? A programmer must rewrite the code.

Machine Learning

MACHINE LEARNING:

```

      +-----+
Input Data -----> |         |
                    | Model  | ---> Learned Rules +
      +-----+
Predictions
Correct Output (Labels) -----> |         |
                                +-----+
```

Example: Predicting Student Grades from ML

Training Data:

```

+-----+-----+-----+-----+
| Score | Study Hours | Attendance | Grade |
+-----+-----+-----+-----+
| 85    | 6 hrs/day   | 95%       | A     |
| 60    | 2 hrs/day   | 70%       | C     |
| 92    | 8 hrs/day   | 98%       | A+    |
+-----+-----+-----+-----+
```

The model studies these examples and **LEARNS** the patterns itself.

New Input: Score=78, Study=5hrs, Attendance=88% → Model predicts: "B"

No programmer wrote "IF score=78 AND study=5hrs THEN grade=B".

The model discovered this pattern from examples.

The Chef Analogy

Think of it this way:

- **Traditional Programming** = Giving a chef a strict recipe book. Every dish must follow the exact steps written by the recipe author. The chef cannot adapt if an ingredient is missing.
- **Machine Learning** = Showing a chef 1,000 photos of finished dishes without any recipes. The chef studies them, identifies patterns, and figures out the underlying recipes themselves. The chef can then improvise with new ingredients.

Pause & Think — 6.3

You are building a system to detect whether a student is at risk of failing. You try traditional programming first: you write rules like "IF attendance < 60% THEN at-risk."

What are **two problems** with this approach? Why might machine learning work better here?

(Hint: Think about students who attend but still fail. Think about students who miss classes but still pass because they study independently.)

Quick Recap

Machine learning enables systems to learn patterns from data automatically, rather than following rules written by a programmer. This makes it powerful for complex, real-world problems where writing every rule manually is impossible.

6.4 Machine Learning Mechanisms

Three Ways a Machine Can Learn

The Hook: Three Ways Students Learn

Imagine three different students learning a new subject:

- **Student A (Supervised):** Studies using flashcards. Each card has a question on the front and the correct answer on the back. They check their answer every time. They learn by correcting their mistakes.
- **Student B (Unsupervised):** Gets handed a mixed pile of 500 unsorted Lego bricks with no labels or instructions. They start sorting by shape, colour, and size — figuring out the groupings themselves, with no one telling them what to call each group.
- **Student C (Reinforcement):** Learns to ride a bike. Every time they balance correctly, they feel the reward of moving forward. Every time they fall, that is a penalty. Over hundreds of tries, they learn exactly how to balance — not from a textbook, but from direct experience.

These three students represent the three main types of machine learning.

Supervised Machine Learning

The Explanation

Supervised machine learning trains a model using **labelled data** — data where every input already has a known, correct output attached to it.

The model learns by comparing its predictions to the correct answers, measuring how wrong it was (the **error**), and adjusting itself to reduce that error. This process repeats thousands of times until the model's predictions become accurate.

"Supervised" means a teacher is guiding the learning process — the correct labels act as the teacher.

What is Labelled Data?

LABELLED DATA EXAMPLES:

Input Features	Label (Output)	Task
Email text content	Spam / Not Spam	Email classification
Medical scan image	Cancer / Normal	Disease detection
Photo of an animal	Cat / Dog / Bird	Image classification
Student study hours	Pass / Fail	Grade prediction
House features	Price in Rs.	House price prediction

How Supervised Learning Works (Step by Step)

STEP 1: Collect labelled training data

Study Hours	Attendance	Grade	
2	60%	Fail	<-- Label already known
5	80%	Pass	<-- Label already known
8	95%	A	<-- Label already known

STEP 2: Feed data into the model. Model makes a prediction.

Model sees: Study Hours=5, Attendance=80% → Predicts: "Pass"

Correct label: "Pass" ✓ Error = 0

STEP 3: When the model is wrong, it adjusts.

Model sees: Study Hours=2, Attendance=60% → Predicts: "Pass"

Correct label: "Fail" ✗ Error is HIGH → Model adjusts its weights

STEP 4: Repeat thousands of times. Error decreases. Accuracy increases.

STEP 5: Use the trained model on NEW, unseen data.

New student: Study Hours=4, Attendance=75% → Model predicts: "Pass"

Two Main Types of Supervised Learning Tasks

- **Classification:** The output is a **category** (e.g., Spam/Not Spam, Pass/Fail, Cat/Dog). The model assigns the input to one of several predefined classes.
- **Regression:** The output is a **number** (e.g., predict the exam score, predict the house price). The model predicts a continuous numerical value.

Real-World Applications of Supervised Learning

- Email spam detection
- Face recognition on your phone
- Medical diagnosis from scans

- Credit card fraud detection
- Predicting whether a student will pass or fail

Pause & Think — 6.4a

You want to build a supervised learning model to predict whether a patient has diabetes.

What features (input columns) would you include in your dataset? What would the label (output) be? What type of task is this — classification or regression?

Quick Recap

Supervised learning uses labelled data (inputs with known correct outputs) to train a model. The model learns by correcting its mistakes. It handles two main tasks: classification (predicting a category) and regression (predicting a number).

Unsupervised Machine Learning

The Explanation

Unsupervised machine learning works with **unlabelled data** — data that has no pre-assigned correct answers. The model must discover structure and patterns entirely on its own.

"**Unsupervised**" means there is no teacher providing correct answers. The model explores freely.

What is Unlabelled Data?

UNLABELLED DATA:

Study Hours	Attendance	???
2	60%	(unknown)
5	80%	(unknown)
8	95%	(unknown)
1	45%	(unknown)
7	90%	(unknown)

No labels. No "Pass" or "Fail". The model must find groupings itself.

The Most Common Unsupervised Technique: Clustering

Clustering groups similar data points together based on their features, without being told what the groups should be called.

CLUSTERING RESULT (the model discovered 3 groups on its own):

CLUSTER A (High performers): Study Hours=7-8, Attendance=90-98%

CLUSTER B (Average performers): Study Hours=4-6, Attendance=70-85%

CLUSTER C (At-risk students): Study Hours=1-2, Attendance=40-65%

The model found these groups WITHOUT being told they existed.

A human then gives these clusters meaningful names.

Real-World Applications of Unsupervised Learning

- **Customer segmentation:** A business discovers that their customers naturally fall into 4 different groups (budget shoppers, luxury buyers, deal hunters, brand loyalists) — without anyone telling the model what those groups should be.
- **Anomaly detection:** A bank finds unusual transaction patterns that do not fit any normal customer cluster — potentially fraudulent transactions.
- **Document grouping:** A search engine groups thousands of news articles by topic without human labelling.
- **Gene expression analysis:** Scientists discover previously unknown genetic patterns in medical data.

Pause & Think — 6.4b

A music streaming service has listening data for 1 million users — what songs they played, how long they listened, what time of day, and what devices they used. They want to group users with similar listening habits together to give better recommendations.

Is this supervised or unsupervised learning? Why? What might the clusters look like?

Quick Recap

Unsupervised learning finds patterns in data without any correct labels or guidance. Clustering is the most common technique — it groups similar data points together. This is useful for discovering hidden structures that humans did not know existed.

Reinforcement Machine Learning

The Explanation

Reinforcement machine learning is a type of learning where an **agent** (the model) learns by taking **actions** in an **environment** and receiving **rewards** (positive feedback) or **penalties** (negative feedback).

The agent does not learn from a dataset of examples. It learns by **doing** — trying things, seeing what works, and doing more of what gets rewarded.

"**Reinforcement**" means the correct behaviour is reinforced through rewards.

The Dog Training Analogy

REINFORCEMENT LEARNING = TRAINING A DOG

Environment = The room

Agent = The dog

Action = What the dog does (sit, bark, roll over)

Reward = Treat

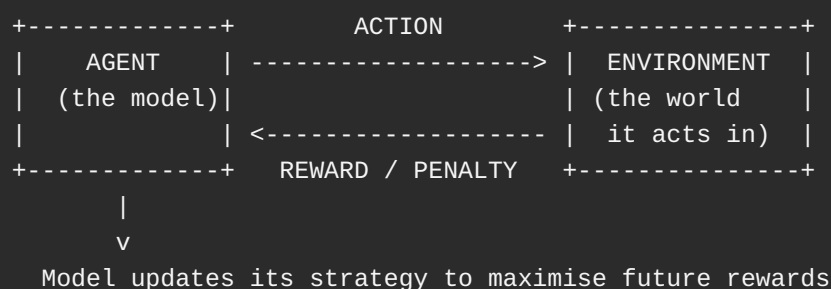
Penalty = "No!"

Dog sits on command → Gets a treat ✓ → Dog learns to sit more

Dog barks randomly → Gets a "No!" ✗ → Dog learns to bark less

Over many trials, the dog learns to maximise treats by doing the right actions.

The Reinforcement Learning Cycle



Real-World Applications of Reinforcement Learning

- **AlphaGo:** Google DeepMind's AI learned to play the board game Go by playing millions of games against itself. It discovered strategies that no human had ever thought of, and beat the world champion.
- **Autonomous cars:** A self-driving car learns to make driving decisions by simulating millions of driving scenarios. Staying in the lane = reward. Crashing = penalty.
- **Robotics:** A robot arm learns to pick up objects by trial and error — millions of simulated attempts where success is rewarded.
- **Game-playing AI:** AI systems like those that play chess, poker, and video games learn through reinforcement.
- **Recommendation systems:** Netflix's recommendation engine learns what to suggest by observing whether users click, watch, or skip — clicks and watches are rewards.

Comparison of All Three Mechanisms

+-----+-----+-----+ +-----+			
Feature	Supervised	Unsupervised	
Reinforcement			
+-----+-----+-----+ +-----+			
Uses labelled data?	Yes	No	No (uses
rewards)			
Needs correct answers?	Yes	No	
No			
Goal	Predict output	Find patterns	Maximise
reward			
Learning style	Teacher guides	Self-discovery	Trial &
error			
Example	Spam detection	Customer clusters	Game-playing
AI			
+-----+-----+-----+ +-----+			

Pause & Think — 6.4c

A game company wants to build an AI that learns to play a new puzzle game from scratch. Which type of machine learning (supervised, unsupervised, or reinforcement) would be most suitable? Why?

Quick Recap

Reinforcement learning trains an agent to make decisions by rewarding correct actions and penalising incorrect ones. It learns through experience, not from a pre-labelled dataset. It powers robotics, game-playing AI, and self-driving vehicles.

6.5 Applications of Machine Learning

The Hook: The \$1 Million Netflix Prize (2006–2009)

In 2006, Netflix had a problem. Their movie recommendation algorithm — the system that suggests what to watch next — was not good enough. Users were not watching enough of the movies it suggested.

So Netflix did something bold: they offered **one million US dollars** to anyone in the world who could improve their recommendation system by just 10%.

Thousands of teams entered the competition. Three years later, in 2009, a team of engineers from around the world — who had never met in person — won the prize by combining multiple machine learning techniques.

The result was a recommendation engine so powerful that today, Netflix estimates that **80% of the content people watch is driven by their recommendation algorithm**, not by what people search for.

That one improvement — powered by machine learning — keeps millions of subscribers and generates billions of dollars in value.

Use of Machine Learning in Healthcare

The Explanation

Healthcare is one of the most impactful areas for machine learning. Human lives depend on correct, fast, and early diagnoses.

HEALTHCARE APPLICATIONS:

+-----+	
+-----+	
Application	How Machine Learning Helps
+-----+	
Cancer detection	Models analyse medical scans (MRI, X-ray, CT)
	and flag abnormal tissue that may be cancerous
+-----+	
Early Alzheimer's detection	Models detect early signs of cognitive decline
	in brain scans years before symptoms appear
+-----+	
Cardiovascular disease	Models predict heart disease risk from patient
prediction	history, blood tests, and vital signs
+-----+	
Drug discovery	Models simulate how drugs interact with cells,
	cutting years off the drug development process
+-----+	
Hospital resource planning	Predictive models forecast patient admission
	rates so hospitals can staff correctly
+-----+	
+-----+	

Why does ML help in healthcare?

A human doctor can study perhaps thousands of patient cases in their career. A machine learning model can analyse millions. It can find patterns too subtle for a human eye to detect consistently.

Use of Machine Learning in Business

The Explanation

Businesses use machine learning to understand customers better, cut costs, and make smarter decisions.

BUSINESS APPLICATIONS:

+-----	
+-----+	
Application	How Machine Learning Helps
+-----	
+-----+	
Customer segmentation	Groups customers by behaviour to target
	marketing more effectively
+-----	
+-----+	
Fraud detection	Detects unusual transaction patterns in real
	time and flags or blocks fraudulent activity
+-----	
+-----+	
Sales forecasting	Predicts future sales based on past trends,
	seasons, and market conditions
+-----	
+-----+	
Chatbots & customer service	AI-powered bots answer customer queries 24/7
	without human agents
+-----	
+-----+	
Product recommendations	Suggests products based on a customer's past
	purchases and browsing behaviour
+-----	
+-----+	
Inventory management	Predicts which products will run low so they
	can be restocked before they run out
+-----	
+-----+	

Use of Machine Learning in Education

The Explanation

Education is being transformed by machine learning tools that personalise learning and support teachers.

EDUCATION APPLICATIONS:

+-----+ +-----+	
Application	How Machine Learning Helps
+-----+ +-----+	
Personalised learning	Adapts content difficulty and pacing to each
	student's individual progress and learning style
+-----+ +-----+	
Student performance tracking	Identifies students at risk of falling behind
	so teachers can intervene early
+-----+ +-----+	
Automated grading	Grades multiple-choice and even short-answer
	questions with high accuracy
+-----+ +-----+	
Intelligent tutoring systems	Provides step-by-step hints and explanations
	when a student is stuck on a problem
+-----+ +-----+	
Plagiarism detection	Compares submitted work against millions of
	documents to identify copied content
+-----+ +-----+	

Everyday Applications of Machine Learning

The Explanation

You already use machine learning dozens of times every day — you just may not have noticed.

YOUR DAILY ML INTERACTIONS:

+-----+-----	
+	
Application	What ML is Doing
+-----+-----	
+	
Face unlock on your phone	Supervised learning recognises your face's
	unique geometric features
+-----+-----	
+	
Voice assistants (Siri,	Speech recognition models convert sound waves
Google Assistant)	into text, then NLP models interpret commands
+-----+-----	
+	
Email spam filter	Naive Bayes or neural network classifies each
	email as spam or not spam before you see it
+-----+-----	
+	
YouTube / Netflix / Spotify	Recommendation engines predict what content
recommendations	you are most likely to enjoy next
+-----+-----	
+	
Google Maps route	Models predict traffic conditions and suggest
suggestions	the fastest route in real time
+-----+-----	
+	
Social media feed	Models decide which posts you are most likely
(Instagram, TikTok)	to engage with and show them first
+-----+-----	
+	
Smart home devices	ML learns your habits (when you wake up, what
(thermostats, lights)	temperature you prefer) and automates them
+-----+-----	
+	

Grab a Partner — 6.5

With a partner, spend 3 minutes listing every application on your phone that you think uses machine learning. Then discuss: **which single application do you think would be most difficult to build? Why?**

Quick Recap

Machine learning is used across healthcare, business, education, and everyday life. From detecting cancer to filtering spam email to recommending music — machine learning is embedded in the tools billions of people use daily.

6.6 Building a Machine Learning Model

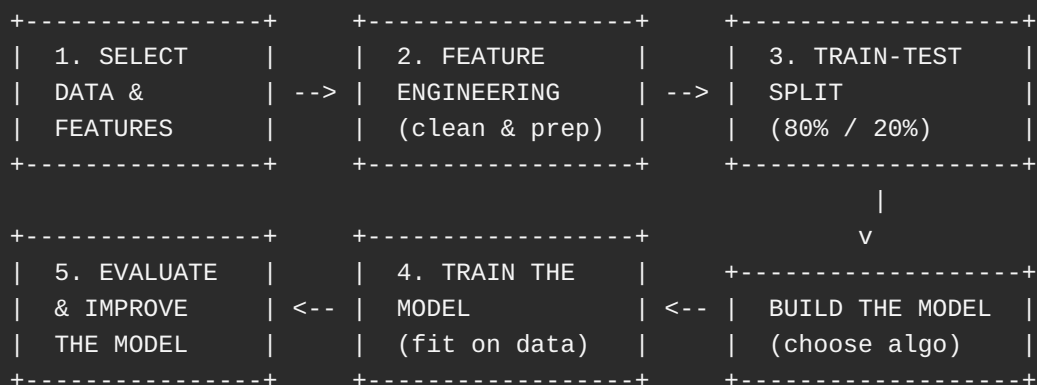
The Practical Process: From Raw Data to Predictions

The Hook

Imagine a doctor who wants to predict which students are likely to fail their final exams before the exams even happen — early enough to give them extra support. The doctor has records of 1,000 past students: their attendance, quiz scores, study hours, assignment grades, and final results.

Building a machine learning model for this is not magic. It is a structured, step-by-step engineering process. Let's walk through every step.

MACHINE LEARNING MODEL BUILDING PROCESS:



Selecting Features and Target Values

The Explanation

Features are the input values you give to the model. Think of them as the clues the model uses to make its prediction.

Target value (also called the label or output) is what the model is trying to predict.

Example: Student Performance Prediction

RAW DATASET:

```
+-----+-----+-----+-----+-----+
+-----+
| Student ID| Study Hours| Attendance %| Quiz Average  | Assignment Score |
Grade      |
+-----+-----+-----+-----+-----+
+-----+
| 001      | 6          | 92          | 78            | 85                |
Pass      |
| 002      | 2          | 55          | 42            | 50                |
Fail      |
| 003      | 8          | 98          | 91            | 93                |
Pass      |
| 004      | 1          | 40          | 35            | 45                |
Fail      |
| 005      | 5          | 75          | 65            | 72                |
Pass      |
+-----+-----+-----+-----+-----+
+-----+
```

FEATURE SELECTION:

- Student ID: REMOVE ✗ (it is just an ID number, tells the model nothing about performance)
- Study Hours: KEEP ✓ (directly related to performance)
- Attendance %: KEEP ✓ (directly related to performance)
- Quiz Average: KEEP ✓ (directly related to performance)
- Assignment Score: KEEP ✓ (directly related to performance)
- Grade: This is the TARGET VALUE ← What we want to predict

AFTER FEATURE SELECTION:

```
Features (X):      +-----+-----+-----+
+-----+
| Study Hours| Attendance %| Quiz Average  | Assignment
Score |
+-----+-----+-----+
+-----+

Target (y):        +-----+
| Grade |
+-----+
```

Why Feature Selection Matters

Bad features add noise and confusion to the model. Imagine adding "Favourite colour" or "Shoe size" as features in a student grade prediction model. These are irrelevant — they would confuse the model and reduce its accuracy.

Good feature selection rule: Include features that have a logical, meaningful relationship to what you are trying to predict.

Quick Recap

Features are the input columns the model learns from. The target is the output the model predicts. Always remove irrelevant columns before training — they add noise and hurt performance.

Feature Engineering Basics

The Explanation

Feature engineering is the process of transforming raw data into a better format so the model can learn from it more effectively.

Raw data is almost never ready to feed directly into a model. It needs to be cleaned, converted, scaled, and sometimes enriched with new calculated columns.

Step 1: Handling Missing Values (Null/Empty Cells)

PROBLEM: Missing data in the dataset

```
+-----+-----+-----+
| Student ID| Study Hours| Attendance %|
+-----+-----+-----+
| 001      | 6         | 92         |
| 002      | ???      | 55         | <-- Missing study hours!
| 003      | 8         | ???      | <-- Missing attendance!
+-----+-----+-----+
```

SOLUTION OPTIONS:

- (a) Drop the row: Only if few rows have missing values
- (b) Fill with mean: Study hours mean = $(6+8)/2 = 7$ → Fill 002 with 7
- (c) Fill with median: Better for data with outliers
- (d) Flag and fill: Add a new column "study_hours_was_missing" = 1/0

Step 2: Encoding Categorical Data (One-Hot Encoding)

Machine learning models work with numbers, not text. If your data has text categories, you must convert them to numbers.

PROBLEM: "School_Type" column has text values

Student ID	School_Type
001	Public
002	Private
003	Public
004	Religious

SOLUTION: One-Hot Encoding

Convert each category into its own binary (0/1) column:

Student ID	School_Public	School_Private	School_Religious
001	1	0	0
002	0	1	0
003	1	0	0
004	0	0	1

Now the model can work with numbers instead of text.

Step 3: Feature Scaling

Different features may have very different value ranges. This can confuse some models.

PROBLEM: Features at very different scales

Student ID	Study Hours	Exam Score
001	6	850
002	2	620
003	8	910

<-- Exam score is 100x bigger!

The model might think Exam Score is more important just because its numbers are bigger. That is wrong.

SOLUTION: Min-Max Scaling (normalise to 0-1 range)

Formula: $(\text{value} - \text{minimum}) / (\text{maximum} - \text{minimum})$

Study Hours scaled: $(6-2)/(8-2) = 4/6 = 0.67$

Exam Score scaled: $(850-620)/(910-620) = 230/290 = 0.79$

Now both features are on the same scale (0 to 1).

Step 4: Creating New Features

Sometimes the most useful feature does not exist in your raw data — you have to create it.

CREATING NEW FEATURES:

Raw features available: "Total Study Hours Per Week" and "Number of School Days"

New feature: $\text{Study_Hours_Per_Day} = \text{Total Study Hours Per Week} / 5$

This new feature might be more predictive of performance than raw weekly hours.

Pause & Think — 6.6a

Your dataset of house prices has these columns:

House_ID	Bedrooms	Bathrooms	House_Colour	Built_Year	Neighbourhood	Price
----------	----------	-----------	--------------	------------	---------------	-------

Which columns would you:

- (a) Keep as features?
- (b) Remove as irrelevant?
- (c) Need to encode (One-Hot)?
- (d) Scale?

What is the target value?

Quick Recap

Feature engineering transforms raw, messy data into a clean, numerical format that models can learn from. It includes handling missing values, encoding text categories as numbers, scaling features to similar ranges, and creating new informative features from existing ones.

Train-Test Split of Data

The Explanation

This is one of the most important concepts in all of machine learning. Read it carefully.

When you build a model, you train it on your dataset. The model studies the data and learns its patterns. But here is the problem: **how do you know if the model truly learned, or if it just memorised the training data?**

The answer: you hide some data from the model during training. Then you use that hidden data to test it.

This is called the **train-test split**.

The Exam Paper Analogy

TRAIN-TEST SPLIT = THE EXAM PREPARATION ANALOGY:

Training Set = Past exam papers (used to study and learn)

Test Set = Brand-new exam questions on exam day (never seen before)

If you only study by memorising the exact questions from past papers without truly understanding the concepts, you will fail the real exam.

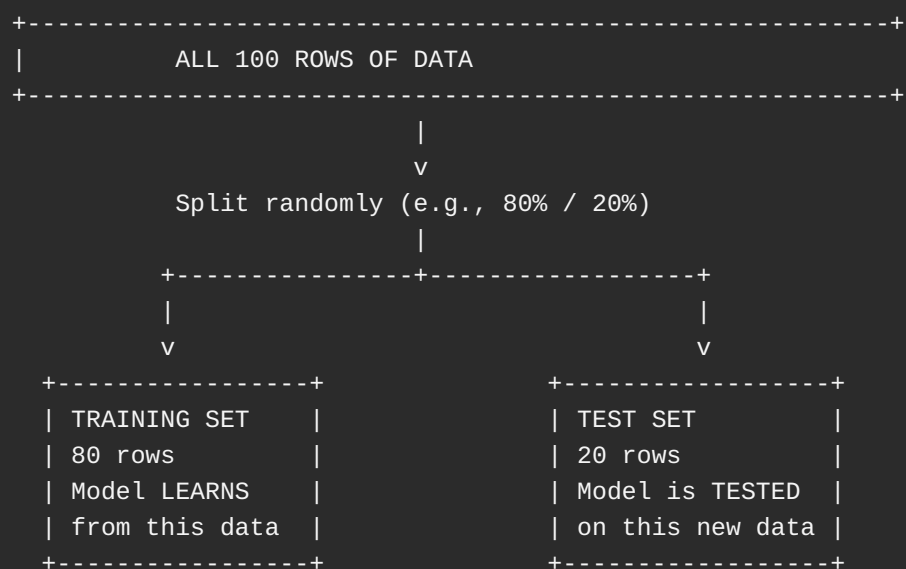
The same is true for a machine learning model.

If a model just memorises training data without truly learning patterns, it will perform badly on new, unseen data.

The test set catches this problem.

How to Split

TOTAL DATASET: 100 student records



The model NEVER SEES the test set during training.

Why 80/20?

The 80/20 split is the most common starting point. But the right split depends on how much data you have:

Dataset Size	Common Split	Reason
Small (<1,000)	70% train / 30% test	Need more test examples
Medium (1,000s)	80% train / 20% test	Standard split
Large (millions)	99% train / 1% test	1% is still thousands

Overfitting vs. Underfitting

Two problems to watch out for:

OVERFITTING:

The model memorises training data.
It performs GREAT on training set.
It performs BADLY on test set.
Like a student who memorises
answers word-for-word without
understanding them.

SOLUTION: More data, simpler model,
regularisation, more features.

UNDERFITTING:

The model is too simple.
It performs BADLY on both
training set and test set.
Like a student who barely
studied at all.

SOLUTION: More training,
more complex model.

The Python Code: Train-Test Split

```
# Import the necessary tools
import pandas as pd
from sklearn.model_selection import train_test_split

# Create a simple student dataset
data = {
    'Study_Hours': [2, 4, 5, 6, 8, 3, 7, 1, 5, 6],
    'Attendance': [55, 70, 80, 85, 95, 60, 90, 40, 78, 88],
    'Quiz_Score': [42, 65, 72, 78, 91, 55, 88, 35, 70, 82],
    'Grade':
['Fail', 'Pass', 'Pass', 'Pass', 'Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Pass']
}

df = pd.DataFrame(data)

# Separate features (X) and target (y)
X = df[['Study_Hours', 'Attendance', 'Quiz_Score']] # Features
y = df['Grade'] # Target

# Split into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2, # 20% goes to test set
    random_state=42 # random_state ensures the same split every time
)

# Check the sizes
print(f"Total records: {len(df)}")
print(f"Training records: {len(X_train)}") # Output: 8
print(f"Testing records: {len(X_test)}") # Output: 2

print("\nTraining Features (X_train):")
print(X_train)

print("\nTraining Labels (y_train):")
print(y_train)
```

Expected output:

```
Total records: 10
Training records: 8
Testing records: 2

Training Features (X_train):
  Study_Hours  Attendance  Quiz_Score
3             6          85           78
7             1          40           35
...

Training Labels (y_train):
3    Pass
7    Fail
...
```

Quick Recap

The train-test split divides your data into two parts: a training set that the model learns from, and a test set that evaluates its performance on unseen data. Never let the model see the test data during training. This prevents the model from simply memorising — it must genuinely learn patterns.

Building a Simple Predictive Model

The Complete Python Walkthrough

Let us build a complete, working linear regression model that predicts student exam scores from study hours. We will follow every step from raw data to final evaluation.

```

# =====
# COMPLETE MACHINE LEARNING WALKTHROUGH: Student Score Predictor
# =====

# STEP 0: Import all necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# STEP 1: Create the dataset
# X = Hours studied (input feature)
# y = Exam score (target value to predict)

X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = np.array([25, 35, 45, 52, 60, 68, 75, 81, 88, 94])

print("Our dataset:")
print("Hours Studied | Exam Score")
print("-" * 30)
for h, s in zip(X.flatten(), y):
    print(f"      {h:2d}      |      {s}")

# STEP 2: Split data into 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"\nTraining set size: {len(X_train)} records")
print(f"Testing set size:   {len(X_test)} records")

# STEP 3: Create and train the Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)  # The model LEARNS from training data

print(f"\nModel learned:")
print(f"  Slope (coefficient): {model.coef_[0]:.4f}")
print(f"  Intercept: {model.intercept_:.4f}")
print(f"  Equation: Exam Score = {model.coef_[0]:.4f} × Hours + {model.intercept_:.4f}")

# STEP 4: Make predictions on the TEST set (unseen data)
y_pred = model.predict(X_test)

print(f"\nPredictions on test data:")
print(f"{'Actual Score':<15} {'Predicted Score':<15}")
print("-" * 30)
for actual, predicted in zip(y_test, y_pred):
    print(f"{actual:<15} {predicted:<15.2f}")

```

```
# STEP 5: Evaluate the model's performance
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"\nModel Performance:")
print(f" Mean Squared Error (MSE): {mse:.4f}")
print(f" R-squared (R²): {r2:.4f}")
print(f" Interpretation: The model explains {r2*100:.1f}% of the variation in exam scores")

# STEP 6: Predict a NEW student's score
new_student_hours = np.array([[7.5]])
predicted_score = model.predict(new_student_hours)
print(f"\nNew prediction: A student who studies 7.5 hours")
print(f" Predicted exam score: {predicted_score[0]:.1f}")
```

Understanding the Output:

Model learned:
Equation: Exam Score = 7.55 × Hours + 16.9

This means: For every extra hour studied,
the predicted score increases by 7.55 points.

Predictions on test data:

Actual Score	Predicted Score
88	85.31
35	32.00

Mean Squared Error (MSE): 10.24
→ The model's predictions are off by about 3.2 points on average (square root of 10.24 ≈ 3.2)

R-squared (R²): 0.972
→ The model explains 97.2% of the variation in scores. That is excellent for a simple model!

Pause & Think — 6.6b

Look at the model equation: **Exam Score = 7.55 × Hours + 16.9**

- If a student studies 0 hours, what score does the model predict? Does this make sense in real life?
- If a student studies 20 hours, the model predicts a score of 168. Maximum possible score is 100. What problem does this reveal about linear regression?
- What feature could you ADD to this model to make it more realistic?

Quick Recap

Building a machine learning model follows clear steps: select features, engineer the data, split into train and test sets, fit the model on training data, make predictions, and evaluate performance. Even a simple linear regression model can produce powerful and interpretable predictions.

6.7 Model Evaluation and Performance Metrics

Why Accuracy Alone Can Be Dangerous

The Hook: The Spam Wars (1990s)

In the early days of email, spam filters used simple rules: "IF the email contains the word FREE, block it."

Spammers fought back. They wrote "F.R.E.E" or "FR33" or "FREEE." The rule-based filters failed immediately.

Engineers switched to machine learning. Instead of writing rules, they trained models on thousands of examples of spam and legitimate emails. The model learned subtle statistical patterns — not just single keywords.

But here is the challenge: **how do you measure if your spam filter is truly good?**

Say your spam filter blocks 100% of spam emails. Sounds perfect! But what if it also blocks 30% of your legitimate emails — including job offers, university admission letters, and important bank notifications?

Accuracy alone would not catch this problem. You need better metrics. That is what this section teaches.

The Confusion Matrix: Your Evaluation Foundation

Before calculating any metric, you need to understand the **confusion matrix** — a table that breaks down every type of correct and incorrect prediction a model makes.

CONFUSION MATRIX for a Binary (Yes/No) Classifier:
(Example: Spam Detector)

		PREDICTED by Model	
		Predicted SPAM	Predicted NOT SPAM
ACTUAL TRUTH	Actual SPAM	TP = 40 (True Positive) Correctly said "Yes, it's spam"	FN = 5 (False Negative) Missed real spam - BAD!
	Actual NOT SPAM	FP = 10 (False Positive) Called a good email spam - BAD	TN = 50 (True Negative) Correctly said "Not spam" - GOOD

UNDERSTANDING THE FOUR CELLS:

TP (True Positive) = 40 → Model said SPAM, it WAS spam. ✓ CORRECT

TN (True Negative) = 50 → Model said NOT SPAM, it was NOT. ✓ CORRECT

FP (False Positive) = 10 → Model said SPAM, but it WASN'T. ✗ WRONG (Type I Error)

FN (False Negative) = 5 → Model said NOT SPAM, but it WAS. ✗ WRONG (Type II Error)

Total predictions = TP + TN + FP + FN = 40 + 50 + 10 + 5 = 105

With TP=40, TN=50, FP=10, FN=5, we can now calculate all metrics.

Accuracy and Error Rate

The Explanation

Accuracy answers: "Out of all predictions the model made, what fraction were correct?"

ACCURACY FORMULA:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Using our spam detector example:

$$\text{Accuracy} = \frac{40 + 50}{40 + 50 + 10 + 5} = \frac{90}{105} = 0.857 = 85.7\%$$

The model correctly classified 85.7% of all emails.

Error Rate is the opposite:

Error Rate = 1 - Accuracy = 1 - 0.857 = 0.143 = 14.3%
14.3% of emails were classified incorrectly.

Why Accuracy Can Be Misleading: The Imbalanced Data Problem

DANGEROUS EXAMPLE: Medical Test for a Rare Disease

Imagine: Only 1% of patients have Disease X.
99% of patients are healthy.

A LAZY MODEL that ALWAYS predicts "Healthy" (never detects the disease):

TP = 0 (never correctly identifies sick patients)
TN = 9900 (correctly identifies all 9,900 healthy patients)
FP = 0 (never wrongly accuses a healthy patient)
FN = 100 (misses all 100 actually sick patients!)

$$\text{Accuracy} = \frac{0 + 9900}{0 + 9900 + 0 + 100} = 99\%$$

THIS MODEL HAS 99% ACCURACY AND IS COMPLETELY USELESS.
It misses every single sick patient. No doctor would use it.

This is why we need Precision, Recall, and F1-Score.

Quick Recap

Accuracy measures the fraction of correct predictions. However, on imbalanced datasets (where one class is much more common), accuracy can be dangerously misleading. Always look at additional metrics.

Precision and Recall

The Explanation

Precision and **Recall** solve the problem that accuracy cannot handle.

Precision

Precision answers: "Of all the cases the model said were POSITIVE (yes), how many were actually positive?"

Precision measures the model's **correctness when it makes a positive prediction**.

PRECISION FORMULA:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Using our spam detector:

$$\text{Precision} = \frac{40}{40 + 10} = 40/50 = 0.80 = 80\%$$

Of all the emails the model called "spam," 80% were actually spam.
The other 20% were legitimate emails wrongly blocked. That is a problem.

High Precision = When the model says "this is spam," it is almost always right.

Low Precision = The model calls too many good emails "spam." Users miss important emails.

Recall

Recall answers: "Of all the actual POSITIVE cases (real spam), how many did the model correctly catch?"

Recall measures the model's **completeness** — did it find all the positive cases?

RECALL FORMULA:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Using our spam detector:

$$\text{Recall} = \frac{40}{40 + 5} = 40/45 = 0.889 = 88.9\%$$

Of all 45 actual spam emails, the model correctly caught 88.9%.
5 spam emails slipped through (False Negatives).

High Recall = The model catches almost all real spam.

Low Recall = The model misses too much spam. Users' inboxes fill up with unwanted messages.

The Precision-Recall Trade-Off

THE CLASSIC TRADE-OFF:

Medical Cancer Detector:

HIGH RECALL is critical. Missing a real cancer case (FN) could cost a life.
We accept more False Positives (some healthy patients flagged for follow-up tests)
as the cost of catching every real cancer case.

Airport Security Scanner:

HIGH RECALL is critical. Missing a real threat (FN) is catastrophic.
We accept more False Positives (innocent bags being searched)
as the cost of catching every real threat.

Email Spam Filter:

HIGH PRECISION is important. Blocking a real job offer (FP) is serious.
We accept more False Negatives (some spam gets through) rather than
risk blocking important emails.

You cannot maximise both at the same time. You must choose
based on which type of error is more costly in your context.

F1-Score

The Explanation

The **F1-Score** gives you a single number that balances precision and recall. It is useful when you need one metric that considers both, especially on imbalanced datasets.

It is called the **harmonic mean** of precision and recall. The harmonic mean punishes extreme values — if either precision or recall is very low, the F1-score will be low too.

F1-SCORE FORMULA:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

STEP-BY-STEP CALCULATION using our spam detector (TP=40, TN=50, FP=10, FN=5):

Step 1: Calculate Precision

$$\text{Precision} = \frac{40}{40 + 10} = 0.8000$$

Step 2: Calculate Recall

$$\text{Recall} = \frac{40}{40 + 5} = 0.8889$$

Step 3: Calculate F1-Score

$$\text{F1-Score} = 2 \times \frac{0.8000 \times 0.8889}{0.8000 + 0.8889} = 2 \times \frac{0.7111}{1.6889} = 2 \times 0.4211 = 0.8421$$

$$\text{F1-Score} = 0.8421 = 84.21\%$$

FINAL SUMMARY OF ALL METRICS:

+-----+-----+		
Metric	Value	
+-----+-----+		
Accuracy	85.71%	
Precision	80.00%	
Recall	88.89%	
F1-Score	84.21%	
+-----+-----+		

The F1-Score of 84.21% tells us the model is reasonably good at balancing catching spam (recall) while not wrongly blocking too many real emails (precision).

When to Use Which Metric

Metric	Use When...
Accuracy	Dataset is balanced (equal classes)
Precision	False Positives are expensive (spam filter, fraud alerts that annoy customers)
Recall	False Negatives are expensive (cancer detection, security screening, fault detection)
F1-Score	You need balance between precision and recall, especially on imbalanced datasets

Grab a Partner — 6.7

Partner A and Partner B each build a disease detection model for a rare illness. Using TP, TN, FP, FN values:

- **Partner A's model:** TP=90, TN=50, FP=5, FN=10
- **Partner B's model:** TP=80, TN=80, FP=2, FN=20

For each model, calculate:

1. Accuracy
2. Precision
3. Recall
4. F1-Score

Then discuss: Which model would you trust more to detect a dangerous disease? Does accuracy alone tell the full story?

Quick Recap

Precision measures how many of the model's positive predictions are correct. Recall measures how many actual positive cases the model found. F1-Score balances both into a single metric. Accuracy alone is misleading on imbalanced datasets — always check all four metrics.

6.8 Validation and Model Improvement

The Hook: Getting the Formula Right

Imagine you bake a cake and it comes out slightly too sweet. You have two choices:

- Bake and sell hundreds of cakes with that same flaw (production errors).
- Taste a small test batch, adjust the sugar, taste again, and only then bake for customers.

The "taste before selling" step is **validation** in machine learning.

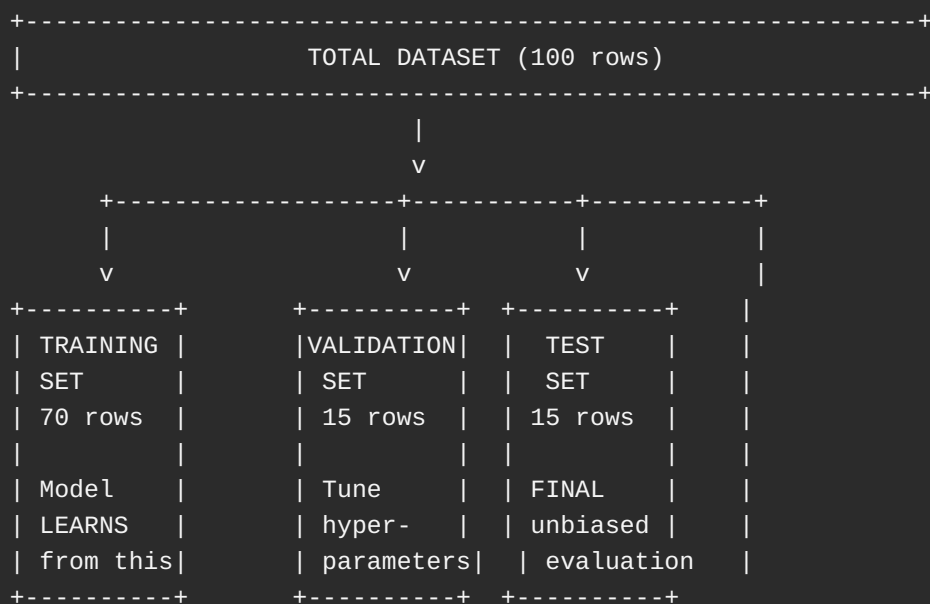
Validation Dataset Preparation

The Explanation

Earlier, we split data into train and test sets (80% / 20%). But there is a problem: if we use the test set to make decisions about the model (like choosing hyperparameters), we are essentially "peeking" at test data. This contaminates our evaluation.

The solution: a **three-way split** — train, validation, and test.

THREE-WAY DATA SPLIT:



Common split: 70% Train / 15% Validation / 15% Test

What each set is used for:

- **Training set:** The model learns patterns from this data.
- **Validation set:** Used to tune hyperparameters and choose the best model. The model does NOT learn from this data, but you use its results to make decisions.
- **Test set:** The FINAL evaluation. Touched only once, at the very end, to get an honest performance measure. Never used for tuning.

Key Principle: The test set must never influence any modelling decisions. If you use the test set to guide improvements, you are lying to yourself about the model's real performance.

Practical Tip: K-Fold Cross-Validation

When data is limited, a single validation split might give misleading results. **K-Fold Cross Validation** is a smarter approach:

K-FOLD CROSS VALIDATION (K=5 example):

Divide training data into 5 equal "folds":

```
+-----+-----+-----+-----+-----+
| Fold1 | Fold2 | Fold3 | Fold4 | Fold5 |
+-----+-----+-----+-----+-----+
```

Round 1: Train on Folds 2,3,4,5 → Validate on Fold 1 → Score = 85%

Round 2: Train on Folds 1,3,4,5 → Validate on Fold 2 → Score = 87%

Round 3: Train on Folds 1,2,4,5 → Validate on Fold 3 → Score = 83%

Round 4: Train on Folds 1,2,3,5 → Validate on Fold 4 → Score = 88%

Round 5: Train on Folds 1,2,3,4 → Validate on Fold 5 → Score = 86%

Average validation score = $(85+87+83+88+86)/5 = 85.8\%$

This gives a MUCH more reliable estimate of model performance than a single validation split.

Quick Recap

A validation set is a portion of data held out from training, used to tune model settings and compare models. The test set is reserved for the final, unbiased evaluation. K-Fold cross-validation gives more reliable estimates when data is limited.

Model Robustness and Reliability

The Explanation

A **robust** model is one that performs well across different situations — not just on the specific examples it was trained on.

A **reliable** model gives consistent results when applied to new data over time.

TESTING FOR ROBUSTNESS:

Robust Model Test:

Test Condition	Fragile Model	Robust Model
Clean, perfect data	92% accuracy	91% accuracy
Data with 10% noise	61% accuracy	88% accuracy
Data from new city	55% accuracy	86% accuracy
Data from next year	48% accuracy	84% accuracy

The fragile model was overfit – it memorised the training data.
The robust model truly learned general patterns.

Signs of a non-robust (fragile, overfit) model:

- Very high accuracy on training data (e.g., 99%) but much lower on test data (e.g., 70%).
- Performance drops dramatically when tested on data from a slightly different source or time period.
- Very sensitive to small changes in input.

How to build more robust models:

1. Collect more diverse training data.
2. Simplify the model (reduce complexity).
3. Use regularisation (a technique that penalises overly complex models).
4. Use cross-validation to test across multiple data splits.

Quick Recap

A robust model performs consistently across different data conditions, not just on the specific examples it was trained on. Robustness is essential for real-world deployment where data is always imperfect and changing.

Improving Models Using Hyperparameter Tuning

The Explanation

Hyperparameters are settings that control how a model learns. They are set by the engineer **before** training begins — the model does not learn them from data.

Parameters (different from hyperparameters) are values the model learns during training (like the slope and intercept in linear regression).

PARAMETERS vs HYPERPARAMETERS:

Parameters (model learns these):

- Linear regression slope and intercept
- Decision tree split thresholds
- Neural network weights

Hyperparameters (engineer sets these before training):

- How deep a decision tree can grow (max_depth)
- How many nearest neighbours to use in KNN (k_neighbors)
- How fast the model learns (learning_rate)
- How many training rounds to run (n_epochs)

The Speaker Volume Analogy

HYPERPARAMETER TUNING = ADJUSTING THE VOLUME OF A SPEAKER

Volume too HIGH → Sound is distorted and unclear (Overfitting)
Model is too complex, memorises training noise

Volume too LOW → Sound is inaudible (Underfitting)
Model is too simple, misses real patterns

Volume just RIGHT → Sound is clear and balanced (Good generalisation)
Model captures real patterns without memorising noise

Your job as a data scientist: find the "just right" setting.

Practical Example: Decision Tree Depth

```
# Example: Finding the best max_depth for a Decision Tree
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
import numpy as np

# Simulated data (use your actual X_train, y_train)
# For illustration, we'll show what the process looks like

depths_to_try = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
validation_scores = []

for depth in depths_to_try:
    # Create a model with this depth setting
    model = DecisionTreeClassifier(max_depth=depth, random_state=42)

    # Use 5-fold cross-validation on the training set
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')

    # Record the average validation accuracy
    avg_score = scores.mean()
    validation_scores.append(avg_score)

    print(f"max_depth={depth:2d} | Validation Accuracy = {avg_score:.3f}")

# Find the best depth
best_depth = depths_to_try[np.argmax(validation_scores)]
print(f"\nBest max_depth: {best_depth}")
print(f"Best validation accuracy: {max(validation_scores):.3f}")
```

What the output looks like:

```
max_depth= 1 | Validation Accuracy = 0.701 (too shallow = underfitting)
max_depth= 2 | Validation Accuracy = 0.752
max_depth= 3 | Validation Accuracy = 0.801
max_depth= 4 | Validation Accuracy = 0.833 <-- sweet spot
max_depth= 5 | Validation Accuracy = 0.829
max_depth= 6 | Validation Accuracy = 0.815
max_depth= 7 | Validation Accuracy = 0.802
max_depth= 8 | Validation Accuracy = 0.788 (getting worse = overfitting)
max_depth= 9 | Validation Accuracy = 0.771
max_depth=10 | Validation Accuracy = 0.760

Best max_depth: 4
```

At depth 4, the model is complex enough to capture real patterns but not so complex that it memorises the training data.

Pause & Think — 6.8

You are tuning a K-Nearest Neighbours (KNN) model. The K value is the number of neighbours the model considers when making a prediction.

- (a) What might happen if $K = 1$? (hint: very sensitive to individual data points)
- (b) What might happen if $K = 1000$ in a dataset of 1,200 records?
- (c) Why do we use the validation set (not the test set) to find the best K?

Quick Recap

Hyperparameters are model settings you choose before training begins. Tuning them — testing different values and comparing validation scores — is how you find the "just right" settings that make a model perform at its best without overfitting or underfitting.

6.9 Predictive Modeling and Causality

The Hook: Ice Cream and Drowning

Every summer, as ice cream sales increase, so does the number of drowning incidents in swimming pools.

A naive data scientist might look at this and say: **"Ice cream causes drowning! We should ban ice cream!"**

This is absurd — and it illustrates one of the most important concepts in all of data science: **the difference between correlation and causation**.

The real explanation? A third hidden variable — **summer heat** — causes both ice cream sales AND swimming pool usage to increase simultaneously. The two events are *correlated* (they rise together), but one does not *cause* the other.

This section teaches you to see past surface patterns and ask the deeper question: **why is this happening?**

Predictive Outcomes in Machine Learning

The Explanation

Predictive modelling uses past data to estimate what might happen in the future. The model does not need to understand *why* — it just needs to identify patterns that reliably predict outcomes.

PREDICTION (Forecasting Future Outcomes):

Training Data Pattern Observed:

Students who studied > 5 hours AND had > 80% attendance

→ Passed their exams 92% of the time

New Student Input:

Study Hours = 6, Attendance = 85%

Model Output:

"PREDICTED RESULT: Pass (Confidence: 87%)"

The model does NOT need to understand WHY studying more leads to passing.
It just needs to find that the pattern exists and is reliable.

Examples of prediction tasks:

- Forecasting tomorrow's weather from today's temperature, humidity, and wind patterns.
- Predicting next month's sales from the past 24 months of sales data.
- Estimating a student's final grade from their midterm scores and attendance.
- Identifying which patients are likely to develop complications after surgery.

Understanding Causality

The Explanation

Causality means that one variable *directly causes* a change in another. This is a deeper, more powerful claim than just saying two variables are correlated (rising and falling together).

CORRELATION vs CAUSATION:

CORRELATION (variables move together – but why?):

Ice cream sales increase in July

Drowning incidents increase in July

┌
└— BOTH caused by HEAT
┐

Correlation ≠ Causation

Just because two variables move together does NOT mean one causes the other.

CAUSATION (one thing directly causes another):

Smoking → Lung cancer

More exercise → Lower blood pressure

More study hours → Better exam performance

Causation IS real. But proving it requires more than just showing two variables are correlated.

How to Establish Causation

Just showing correlation in data is NOT enough. To prove causation, scientists use:

1. **Controlled experiments (Randomised Control Trials):** Randomly assign some participants to one group (e.g., extra tutoring) and others to a control group (no extra tutoring). If the tutored group consistently performs better, the tutoring is likely a cause.
 2. **Temporal precedence:** The cause must happen BEFORE the effect. Studying more must come BEFORE better grades.
 3. **Ruling out other explanations:** Could a third variable explain both? (e.g., student motivation could cause both more studying AND better grades — making motivation the real cause.)
-

Difference Between Prediction and Causation

The Practical Walkthrough

SCENARIO: Student Exam Performance

PREDICTION (What will happen?):

Model finds: students with attendance > 85% AND study hours > 5
tend to score above 70.

New student: Attendance=90%, Study hours=6 hours

Prediction: Score $\approx$ 78

The model does not know WHY high attendance leads to good scores.
It just found a pattern and uses it to forecast.

CAUSATION (Why does it happen?):

Question: Does high attendance CAUSE better scores?

Possible causal chain:

High attendance → More exposure to teaching → Better understanding
→ Higher scores

Possible confound: Student motivation causes BOTH high attendance
AND more studying. So the REAL cause might be motivation, not attendance.

To prove attendance is causal: do an experiment where we randomly
require some students to attend 90%+ and let others attend freely.
If the required-attendance group scores significantly higher, attendance
is likely a cause.

Real-World Importance

Why does this distinction matter?

- **Wrong conclusion:** Schools see that students with smartphones get lower grades. They ban smartphones.
- But: Maybe less engaged students both use phones more AND study less. Banning phones may not improve grades.
- **Correct analysis:** Do a controlled experiment. Give half the students lockable phone pouches for a month. Compare grades to the other half. Now you can measure the causal effect of phone restriction.

KEY EXAMPLES: PREDICTION vs CAUSATION

Correlation / Prediction	Causation
Ice cream sales predict drowning	Heat causes both; no direct causal link
Shoe size predicts reading level in young children	Age causes both; shoes don't teach reading
Students with higher shoe prices get better grades	Wealth causes both; shoes don't cause better grades directly
Rooster crows before sunrise	Rooster doesn't cause the sun to rise
Hospital admissions peak in winter	Cold weather causes illness AND people to seek warmth indoors (spreads illness)

Pause & Think — 6.9

An AI model analyses data from a school and finds: **"Students who bring a water bottle to school score 12% higher on exams."**

- Is this a correlation or a proven cause-and-effect relationship?
- Think of at least TWO possible explanations for this pattern that do NOT involve water bottles directly causing better scores.
- How would you design an experiment to test whether drinking more water actually improves exam performance?

Quick Recap

Prediction uses patterns in past data to estimate future outcomes — it does not require understanding *why*. Causation explains the *why* — it identifies which variables directly influence other variables. Correlation is not causation. Never make policy decisions based on correlation alone without investigating causality.

6.10 Machine Learning Tools and Platforms

Choosing Your Toolkit

The Hook: The Right Tool for the Right Job

A carpenter does not use a hammer to cut wood. A data scientist does not use Excel to train a neural network. Every tool has its strengths and the right context for use.

In this section, you will meet the three most important tools in data science education: Excel, R, and Python. By the end, you will know which tool to use and when.

Using Excel for Data Analysis

The Explanation

Microsoft Excel is a spreadsheet tool that organises data in rows and columns. It is the perfect starting point for beginners because it is visual, interactive, and does not require programming.

What Excel Does Well

EXCEL STRENGTHS:

+-----+-----+-----+		
Feature	What it means for you	
+-----+-----+-----+		
Data Organisation	Enter and view data in clear rows and columns	
Sorting & Filtering	Quickly sort by any column; filter to see	
	only rows that meet a condition	
Formulas & Calculations	=AVERAGE(B2:B100), =SUM(), =MAX(), =MIN()	
Charts & Visualisation	Bar charts, line charts, pie charts, scatter	
	plots – created with a few clicks	
Pivot Tables	Summarise and group data automatically	
+-----+-----+-----+		

Practical Example: Analysing Student Data in Excel

Step 1: Enter your data:

```
+-----+-----+-----+-----+-----+
| Row | Name  | Quiz1 | Quiz2 | Avg   |
+-----+-----+-----+-----+-----+
| 2   | Aisha | 85    | 78    | =AVERAGE(C2:D2) |
| 3   | Bilal | 72    | 80    | =AVERAGE(C3:D3) |
| 4   | Fatima | 91    | 88    | =AVERAGE(C4:D4) |
+-----+-----+-----+-----+-----+
```

Step 2: In cell E2, type: =AVERAGE(C2:D2) → Result: 81.5

Step 3: Drag that formula down to E3 and E4 to calculate all averages automatically.

Step 4: Select columns A and E, insert a Bar Chart to visualise average scores.

Excel Limitations

Excel is great for small datasets. But it struggles with:

- Datasets larger than ~1 million rows
- Complex statistical analysis
- Building machine learning models
- Automation and scripting

When you outgrow Excel, you move to R or Python.

Quick Recap

Excel is the best starting point for data organisation, simple calculations, and basic charts. It is visual and requires no programming knowledge. For larger datasets and machine learning, more powerful tools are needed.

Introduction to R for Data Science

The Explanation

R is a programming language designed specifically for statistical analysis and data visualisation. It is widely used in academic research, medical statistics, and scientific data analysis.

What R Does Well

R STRENGTHS:

Feature	What it means for you
Statistical Analysis	Built-in functions for every statistical test
Data Visualisation	ggplot2 library produces publication-quality charts and graphs
Large Dataset Handling	R handles datasets much larger than Excel
Research & Academic Use	Widely used in scientific papers and research
Packages (libraries)	18,000+ free packages for specialised tasks

A Simple R Example

```
# Load the built-in dataset
data(mtcars)

# Look at the first 6 rows
head(mtcars)

# Calculate basic statistics
mean(mtcars$mpg)    # Average miles per gallon
sd(mtcars$mpg)      # Standard deviation
summary(mtcars)     # Full statistical summary of all columns

# Create a histogram of fuel efficiency
hist(mtcars$mpg,
      main = "Distribution of Car Fuel Efficiency",
      xlab = "Miles Per Gallon",
      col = "steelblue")

# Simple linear regression: Does engine size predict fuel efficiency?
model <- lm(mpg ~ wt, data = mtcars)
summary(model)      # Shows the model coefficients and R-squared
```

R is particularly powerful for statistical work and is the preferred tool in research environments.

Quick Recap

R is a programming language built specifically for statistics and data visualisation. It is widely used in academic research and handles large datasets efficiently. It is more powerful than Excel but requires learning programming syntax.

Python and Jupyter Notebooks for Machine Learning

The Explanation

Python is the most popular programming language for machine learning. It is general-purpose (used for web development, automation, science, and AI), easy to read, and has the richest ecosystem of data science libraries.

Python's Key Libraries for Data Science

PYTHON ECOSYSTEM FOR DATA SCIENCE:

+-----+-----+	
Library	What it does
+-----+-----+	
pandas	Work with data tables (like Excel in Python)
NumPy	Fast numerical calculations and arrays
matplotlib	Create charts and plots
seaborn	Beautiful statistical visualisations
scikit-learn	Machine learning models (regression, classifiers)
TensorFlow/Keras	Deep learning and neural networks
Jupyter Notebook	Interactive environment (code + results + text)
+-----+-----+	

What is Jupyter Notebook?

Jupyter Notebook is an interactive coding environment where you can:

- Write code in **cells**
- Run each cell individually and see the result immediately below it
- Mix code, charts, and explanatory text in one document
- Share your entire analysis as a readable document

JUPYTER NOTEBOOK STRUCTURE:

```
+-----+
| [Markdown cell] # Chapter 6 Analysis |
| This notebook analyses student performance data. |
+-----+
| [Code cell] |
| import pandas as pd |
| df = pd.read_csv('students.csv') |
| df.head() |
+-----+
| [Output cell] |
| ID | Name | Score | Grade |
| 1 | Aisha | 85 | A |
| 2 | Bilal | 72 | B |
+-----+
| [Code cell] |
| df['Score'].hist() |
+-----+
| [Output cell] |
| [Histogram chart displays here] |
+-----+
```



```

# =====
# COMPLETE PYTHON DATA SCIENCE WORKFLOW
# Working with student performance data
# =====

# 1. IMPORT LIBRARIES
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, confusion_matrix)

# 2. LOAD AND INSPECT DATA
# Creating sample data (in real projects, you'd load a CSV file)
data = {
    'Study_Hours': [2, 4, 6, 1, 7, 3, 8, 5, 2, 6, 4, 7, 1, 5, 8],
    'Attendance_Pct': [55, 70, 88, 40, 92, 62, 96, 78, 50, 85, 73, 94, 45, 80, 97],
    'Quiz_Average': [40, 62, 80, 30, 88, 55, 92, 70, 42, 81, 65, 90, 35, 74, 93],
    'Grade':
['F', 'P', 'P', 'F', 'P', 'F', 'P', 'P', 'F', 'P', 'P', 'P', 'F', 'P', 'P']
}

df = pd.DataFrame(data)

print("=== DATA OVERVIEW ===")
print(f"Total records: {len(df)}")
print(f"\nData sample:")
print(df.head())
print(f"\nClass distribution:")
print(df['Grade'].value_counts())
print(f"\nBasic statistics:")
print(df.describe())

# 3. PREPARE FEATURES AND TARGET
X = df[['Study_Hours', 'Attendance_Pct', 'Quiz_Average']] # Features
y = df['Grade'] # Target

# 4. SPLIT DATA
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"\n=== DATA SPLIT ===")
print(f"Training samples: {len(X_train)}")
print(f"Testing samples: {len(X_test)}")

# 5. SCALE FEATURES (important for KNN which uses distances)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

```

```

X_test_scaled = scaler.transform(X_test)

# 6. TRAIN THE MODEL
model = KNeighborsClassifier(n_neighbors=3) # K=3 is our hyperparameter
model.fit(X_train_scaled, y_train)

# 7. MAKE PREDICTIONS
y_pred = model.predict(X_test_scaled)

# 8. EVALUATE
print("\n=== MODEL EVALUATION ===")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred, pos_label='P'):.4f}")
print(f"Recall: {recall_score(y_test, y_pred, pos_label='P'):.4f}")
print(f"F1-Score: {f1_score(y_test, y_pred, pos_label='P'):.4f}")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# 9. PREDICT A NEW STUDENT
new_student = scaler.transform([[5.5, 82, 75]]) # 5.5 hrs, 82%, quiz avg 75
prediction = model.predict(new_student)
print(f"\n=== NEW PREDICTION ===")
print(f"New student (5.5 hrs study, 82% attendance, 75 quiz avg)")
print(f"Predicted grade: {prediction[0]}")

```

Python vs. R vs. Excel — When to Use What

Task	Best Tool	Why	Notes
Quick data viewing	Excel	Visual, no code	Limited by size
Basic charts	Excel	Easy, visual	
Statistical analysis	R	Built for stats	Research standard
Academic research	R	Wide acceptance	
Machine learning	Python	Best ML libraries	Industry standard
Deep learning / AI	Python	TensorFlow/PyTorch	
Web scraping data	Python	requests, bs4	
Automation tasks	Python	General purpose	
Data cleaning large	Python (pandas)	Fast, scalable	
Teaching beginners	Excel → Python	Gradual learning	Start with Excel

Pause & Think — 6.10

You have been asked to analyse patient records from a hospital to predict which patients are at risk of readmission within 30 days of discharge.

- (a) Which tool (Excel, R, or Python) would you use for this task? Why?
- (b) Is this a classification or regression problem? Why?
- (c) What features (columns) would you expect to find in this dataset that would be useful for your model?

Quick Recap

Python is the industry standard for machine learning, with powerful libraries like pandas, scikit-learn, and matplotlib. Jupyter Notebook provides an interactive, visual environment for writing and sharing data science analyses. Excel is ideal for beginners and quick analyses. R is preferred for statistical research. Most data scientists know all three.

Chapter Summary

You have just completed one of the most important chapters in modern computing. Let us bring everything together.

CHAPTER 6 – KEY CONCEPTS AT A GLANCE:

DATA SCIENCE

The process of collecting, cleaning, analysing, and interpreting data to find patterns and support decision-making.

It is interdisciplinary: maths + stats + coding + domain knowledge

|
v

DATA TYPES

Structured = organised tables (easy to analyse)

Unstructured = images, audio, video, text (needs special tools)

|
v

MACHINE LEARNING

Systems that learn patterns from data automatically.

Supervised → uses labelled data (classification & regression)

Unsupervised → finds hidden patterns in unlabelled data

Reinforcement → learns from rewards and penalties

|
v

MODEL BUILDING

Select features → Engineer data → Train-Test Split

→ Train model → Evaluate → Tune hyperparameters → Deploy

|
v

EVALUATION METRICS

Accuracy = $(TP+TN) / \text{Total}$ [misleading on imbalanced data]

Precision = $TP / (TP+FP)$ [correctness of positive calls]

Recall = $TP / (TP+FN)$ [completeness – catching all +ves]

F1-Score = $2 \times (P \times R) / (P+R)$ [balance of Precision & Recall]

|
v

PREDICTION vs CAUSATION

Prediction = What will happen? (uses patterns in past data)

Causation = Why does it happen? (requires experiments to prove)

Correlation ≠ Causation. Always investigate the WHY.

|
v

TOOLS

Excel	→	Beginners, quick analysis, small datasets
R	→	Statistical analysis, academic research
Python	→	Machine learning, large datasets, industry standard

Key Vocabulary Glossary

Term	Plain-English Definition
Data Science	The field that turns raw data into useful knowledge using maths, statistics, and coding.
Structured Data	Data organised in neat rows and columns, like a spreadsheet.
Unstructured Data	Data without a fixed format — images, audio, videos, text messages.
Machine Learning	A type of AI where computers learn patterns from data instead of following written rules.
Supervised Learning	ML with labelled data — the model learns from examples that have correct answers.
Unsupervised Learning	ML without labels — the model discovers hidden patterns and groups on its own.
Reinforcement Learning	ML through trial and error — rewards guide the model toward better decisions.
Feature	An input column used by the model (e.g., study hours, attendance).
Target Value	The output the model tries to predict (e.g., exam grade).
Feature Engineering	The process of cleaning, scaling, encoding, and creating features for better model performance.
One-Hot Encoding	Converting text categories into binary (0/1) columns so models can use them.
Feature Scaling	Adjusting feature values to the same scale so no feature dominates unfairly.
Train-Test Split	Dividing data into a training portion (model learns) and a testing portion (model is evaluated).
Overfitting	When a model memorises training data and performs poorly on new, unseen data.
Underfitting	When a model is too simple and fails to capture the patterns even in training data.
Confusion Matrix	A table showing TP, TN, FP, and FN — the foundation for all evaluation metrics.
Accuracy	Fraction of all predictions that were correct: $(TP+TN) / \text{Total}$.
Precision	Of all positive predictions, what fraction were actually positive: $TP / (TP+FP)$.
Recall	Of all actual positives, what fraction did the model correctly identify: $TP / (TP+FN)$.

Term	Plain-English Definition
F1-Score	Harmonic mean of Precision and Recall — balances both into one number.
True Positive (TP)	Model predicted positive and it was correct.
True Negative (TN)	Model predicted negative and it was correct.
False Positive (FP)	Model predicted positive but it was wrong (Type I Error).
False Negative (FN)	Model predicted negative but it was wrong (Type II Error).
Validation Set	A portion of data used to tune hyperparameters — separate from both train and test sets.
Hyperparameter	A model setting chosen by the engineer before training (e.g., max_depth, k_neighbors).
Hyperparameter Tuning	Testing different hyperparameter values to find the setting that gives the best validation performance.
Robustness	A model's ability to perform consistently well across different data conditions.
Prediction	Estimating what will happen in the future based on past data patterns.
Causality	A cause-and-effect relationship — one variable directly causes a change in another.
Correlation	Two variables that move together — but this does not mean one causes the other.
Regression	A supervised learning task where the output is a continuous number.
Classification	A supervised learning task where the output is a category.
Clustering	An unsupervised technique that groups similar data points together.
Jupyter Notebook	An interactive coding environment that mixes code, output, and text in one document.
pandas	A Python library for working with data tables (like Excel in code).
scikit-learn	A Python library with ready-to-use machine learning algorithms.

Exercises

Multiple Choice Questions

1. Data science is mainly concerned with:

- (a) Designing hardware
- **(b) Extracting useful information from data ✓**
- (c) Writing only code
- (d) Creating websites

2. Data organised in rows and columns is called:

- (a) Unstructured data
- **(b) Structured data ✓**
- (c) Raw data
- (d) Random data

3. An example of unstructured data is:

- (a) Tables
- (b) Spreadsheets
- **(c) Images and videos ✓**
- (d) Databases

4. Machine learning allows systems to:

- (a) Work without data
- **(b) Learn from data and improve performance ✓**
- (c) Only store data
- (d) Replace databases

5. The type of machine learning that uses labelled data is:

- (a) Unsupervised learning
- (b) Reinforcement learning
- **(c) Supervised learning ✓**
- (d) Rule-based learning

6. The machine learning method that learns through rewards and penalties is:

- (a) Supervised learning
- (b) Unsupervised learning
- **(c) Reinforcement learning ✓**
- (d) Rule-based learning

7. Feature engineering is used to:

- (a) Delete data
- **(b) Improve data for better model performance ✓**
- (c) Store data
- (d) Visualise data

8. The metric that measures the overall correctness of a model is:

- (a) Precision
- (b) Recall
- **(c) Accuracy ✓**
- (d) F1 score

9. Train-test split is used to:

- (a) Delete data
- **(b) Divide data for training and testing ✓**
- (c) Store data
- (d) Visualise data

10. A commonly used tool for machine learning and data analysis is:

- (a) MS Word
 - **(b) Python ✓**
 - (c) PowerPoint
 - (d) Notepad
-

Short Answer Questions

1. What is data science?

Data science is the interdisciplinary field that collects, cleans, analyses, and interprets data to discover patterns and useful information, supporting evidence-based decision-making. It combines mathematics, statistics, computer science, and domain knowledge.

2. What are the two main types of data?

The two main types are structured data (organised in fixed rows and columns, like spreadsheets) and unstructured data (free-form data like images, audio, video, and text).

3. What is the difference between structured and unstructured data?

Structured data has a fixed, organised format that is easy to search and analyse directly.

Unstructured data has no fixed format and requires special tools (like image recognition or natural language processing) to process.

4. Name two common data collection methods.

Surveys (questionnaires filled out by people) and sensors (devices that automatically collect measurements like temperature or GPS location).

5. What is machine learning?

Machine learning is a subfield of AI where systems learn to make predictions or decisions by finding patterns in data, rather than following rules written explicitly by a programmer.

6. What is supervised learning?

Supervised learning is a type of machine learning that trains a model using labelled data — data where each input already has a known correct output. The model learns by comparing its predictions to the correct answers and reducing its errors.

7. What is the purpose of feature selection in machine learning?

Feature selection identifies which input columns (features) are relevant and meaningful for predicting the target value. Removing irrelevant features reduces noise and improves model accuracy.

8. What does accuracy measure in a model?

Accuracy measures the percentage of all predictions that the model got correct: (True Positives + True Negatives) divided by the total number of predictions.

9. What is the difference between prediction and causality?

Prediction estimates future outcomes based on patterns in past data, without necessarily explaining why those patterns exist. Causality identifies a direct cause-and-effect relationship between variables. A model can make accurate predictions without understanding causality.

10. Name any one tool used for machine learning.

Python (with libraries like scikit-learn, pandas, and NumPy) is the most widely used tool for machine learning.

Long Questions

1. Explain the concept of data science and discuss its importance in modern applications.

Data science is the process of collecting raw data from various sources, cleaning and organising it, analysing it using statistical and computational tools, and interpreting the results to extract meaningful knowledge. It is called interdisciplinary because it draws on mathematics, statistics, computer science, and domain-specific expertise.

Its importance in modern life cannot be overstated. In healthcare, data science enables early disease detection and personalised treatment plans. In business, it drives customer segmentation, sales forecasting, and fraud detection. In education, it personalises learning and identifies at-risk students. In everyday life, it powers the recommendation algorithms on YouTube and Netflix, the spam filters in your email, and the navigation system in Google Maps.

The modern economy is built on data-driven decisions. Organisations that fail to use data science effectively fall behind competitors who do. Data science is not just a technical skill — it is a fundamental literacy for the 21st century.

2. Describe different types of data and explain various data sources and collection methods.

There are two fundamental types of data:

Structured data is organised in a fixed format of rows and columns. Each row is one record; each column is one attribute. Examples include student grade sheets, bank transaction records, and hospital patient databases. Structured data is straightforward to search, sort, filter, and feed into machine learning models.

Unstructured data has no fixed format. It includes emails, photos, audio recordings, videos, social media posts, and handwritten notes. It is far more abundant than structured data (making up over 80% of all data) but requires specialised tools to process — such as image recognition, speech-to-text, and natural language processing.

Data can be collected through many methods. **Surveys and questionnaires** gather opinions and information directly from people. **Sensors and IoT devices** automatically measure and record physical properties like temperature, speed, and location. **Web scraping** uses software to automatically collect data from websites. **Transaction systems** automatically log every purchase, click, or action. **Interviews** capture qualitative information through conversation.

Collections can be **manual** (human effort) or **automatic** (machines and software), and either **real-time** (captured and processed immediately) or **batched** (saved over time and analysed later).

3. Explain the concept of machine learning and describe its different mechanisms.

Machine learning enables computer systems to learn patterns from data and improve their performance without being explicitly programmed for every scenario.

Supervised machine learning trains on labelled data — each input has a known correct output. The model learns by correcting its mistakes, comparing predictions to correct answers. It handles classification tasks (predicting categories like "spam/not spam") and regression tasks (predicting numbers like exam scores). Examples include email spam detection, disease diagnosis from medical scans, and exam result prediction.

Unsupervised machine learning works with unlabelled data. The model discovers hidden structures and patterns without guidance. Clustering — grouping similar data points together — is the most common technique. Examples include customer segmentation, document grouping, and anomaly detection.

Reinforcement machine learning trains an agent to make decisions by rewarding correct actions and penalising incorrect ones. The agent learns through repeated trial and error in an environment. It does not use a labelled dataset — it learns from direct experience. Examples include game-playing AI (AlphaGo), autonomous vehicles, and robotic control systems.

Each mechanism has strengths suited to different types of problems. Choosing the right mechanism is one of the first and most important decisions in any machine learning project.

4. Discuss the applications of machine learning in healthcare, business, education, and daily life.

Healthcare: Machine learning analyses medical images (X-rays, MRIs, CT scans) to detect cancer, Alzheimer's disease, and cardiovascular conditions earlier than traditional methods allow. It predicts which patients are likely to develop complications after surgery. It accelerates drug discovery by simulating how molecules interact with biological systems.

Business: Companies use machine learning for customer segmentation (grouping customers by behaviour for targeted marketing), fraud detection (identifying unusual transaction patterns in real time), sales forecasting (predicting future demand from historical data), and chatbots (providing automated 24/7 customer service).

Education: Machine learning personalises learning content to each student's pace and learning style. It tracks student performance over time and identifies at-risk students before they fall behind. It automates grading and plagiarism detection, freeing teachers to focus on instruction.

Daily Life: Face recognition unlocks phones. Voice assistants (Siri, Google Assistant) understand spoken commands. Email spam filters block unwanted messages. Navigation apps predict the fastest route. Streaming services (YouTube, Spotify, Netflix) recommend content. Social media feeds show the most engaging posts first. Smart home thermostats learn your temperature preferences automatically.

5. Explain the steps involved in building a machine learning model, including feature selection, feature engineering, and train-test split.

Building a machine learning model involves six clear steps:

Step 1 — Data Collection: Gather raw data from relevant sources (databases, sensors, surveys, APIs). The quality and quantity of this data determines the upper limit of model performance.

Step 2 — Feature Selection: Identify which columns (features) are meaningful for predicting the target value. Remove irrelevant columns (like Student ID or favourite colour in a grade prediction model) that add noise without adding information. The target value — what you want to predict — is separated from the features.

Step 3 — Feature Engineering: Clean and transform raw data so models can learn from it. Handle missing values (fill with mean/median or drop rows), encode text categories as binary (0/1) columns using One-Hot Encoding, scale numerical features to similar ranges, and create new informative features from existing ones.

Step 4 — Train-Test Split: Divide the data into a training set (the model learns from this — typically 80%) and a test set (held back for final evaluation — typically 20%). The model must never see the test set during training. This ensures the evaluation reflects how the model will perform on truly new, unseen data.

Step 5 — Model Training: Fit the chosen machine learning algorithm on the training data. The model adjusts its internal parameters to minimise prediction errors on the training set.

Step 6 — Evaluation and Improvement: Test the model on the test set and calculate metrics (accuracy, precision, recall, F1-score). If performance is unsatisfactory, tune hyperparameters using the validation set, add more data, or try a different algorithm.

6. Describe model evaluation techniques and explain accuracy, precision, recall, and F1-score.

Model evaluation measures how well a model's predictions match reality on unseen test data. The foundation is the **confusion matrix**, which categorises every prediction as TP (correctly predicted positive), TN (correctly predicted negative), FP (incorrectly predicted positive), or FN (incorrectly predicted negative).

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$. It measures the overall fraction of correct predictions. It is simple and intuitive but misleading on imbalanced datasets — a model that always predicts the majority class can achieve high accuracy while being completely useless.

Precision = $TP / (TP + FP)$. It answers: of all the cases the model called positive, how many actually were? High precision is important when false positives are costly (e.g., a spam filter incorrectly blocking important emails).

Recall = $TP / (TP + FN)$. It answers: of all the actual positive cases, how many did the model find? High recall is critical when false negatives are costly (e.g., a cancer detection model missing real cancer cases).

F1-Score = $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$. It is the harmonic mean of precision and recall, providing a single balanced metric. It is particularly useful for imbalanced datasets where one class is much rarer than the other.

7. Explain validation in machine learning and discuss how models can be improved using parameter tuning.

Validation is the process of evaluating a model's performance on data it has not been trained on, before committing to its final evaluation on the test set. A separate **validation set** is held out from training data for this purpose. The model is trained on the training set, evaluated on the validation set, adjusted based on those results, and only finally evaluated on the test set.

When data is limited, **K-Fold Cross Validation** provides more reliable estimates by training and validating the model K times, each time on a different portion of the data, and averaging the results.

Hyperparameter tuning improves model performance by finding the best values for the settings that control how the model learns. Unlike parameters (which the model learns automatically during training), hyperparameters are set by the engineer. Examples include the maximum depth of a decision tree, the number of neighbours (K) in KNN, and the learning rate in neural networks.

The tuning process tests multiple hyperparameter values (e.g., `max_depth = 1, 2, 3, 4, ... 10`), evaluates each setting's performance on the validation set, and selects the value that achieves the best validation performance. This is like adjusting the volume of a speaker: too low and the model underfits, too high and it overfits — the goal is finding the setting where performance is just right.

8. Discuss predictive modeling and causality, and explain the importance of model interpretation in real-world use cases.

Predictive modelling uses machine learning to estimate future outcomes based on patterns learned from historical data. A model trained on past student records can predict whether a new student is at risk of failing. A model trained on weather data can predict tomorrow's rainfall. The model does not need to understand *why* these patterns exist — it only needs to find that they are reliable and consistent.

Causality goes deeper. It identifies the actual cause-and-effect mechanisms behind observed patterns. Ice cream sales and drowning rates are correlated (both rise in summer), but eating ice cream does not cause drowning. The hidden causal variable is summer heat, which causes both simultaneously. A predictive model might successfully use ice cream sales to predict drowning risk, but this would be a meaningless and misleading causal story.

In real-world applications, **model interpretation** is critical:

- **Healthcare:** A model predicts high readmission risk for certain patients. Doctors need to understand *which features* drive this prediction (was it the medication, the age, the diagnosis?) to intervene meaningfully.
- **Education:** A model flags students as at-risk. Teachers need to understand whether the cause is attendance, study time, or home circumstances — so they can address the right root cause.

- **Finance:** A model denies a loan application. Regulations in many countries require that decisions be explainable to the applicant — "the algorithm said no" is not acceptable.

Responsible data science requires understanding not just *what* the model predicts, but *why* it makes those predictions, and whether those patterns reflect genuine causal relationships or superficial correlations that could lead to wrong actions.

Quick Review Cards

Cut out or photograph these cards for last-minute revision:

SUPERVISED LEARNING Uses: Labelled data Goal: Predict a known output type Tasks: Classification, Regression Examples: Spam detection, disease diagnosis	UNSUPERVISED LEARNING Uses: Unlabelled data Goal: Discover hidden patterns Tasks: Clustering, Association Examples: Customer grouping, document clustering
REINFORCEMENT LEARNING Agent takes actions Environment gives reward/penalty Goal: Maximise total reward Examples: AlphaGo, self-driving cars, robotics	EVALUATION METRICS Accuracy = $(TP+TN)/Total$ Precision = $TP/(TP+FP)$ Recall = $TP/(TP+FN)$ F1 = $2 \times P \times R / (P+R)$
TRAIN-TEST SPLIT Train (80%): Model LEARNS Test (20%): Model is EVALUATED NEVER show test data during training Overfitting = memorising not learning Validation set: tunes hyperparams Test set: final unbiased evaluation	PREDICTION vs CAUSATION Prediction: What will happen? (uses patterns in past data) Causation: Why does it happen? (requires controlled experiments) Correlation $\neq$ Causation (ice cream $\neq$ drowning)

A Final Word From Your Guide

You have just walked through the foundations of data science and machine learning — the same foundations that power the most impactful technologies of our time.

The tools will change. New libraries will emerge. Models will become more powerful. But the concepts you learned here — clean your data, split your sets, measure all your metrics, never confuse correlation with causation — these will remain true for decades.

You are not just a student of computer science. You are a future builder of the intelligent systems that will shape your world.

Now go build something extraordinary.